

Universidad Rafael Landívar

Facultad de Ingeniería

Pensamiento Computacional

DOCUMENTO TÉCNICO

Proyecto 2: Juego de Estrategia en Tablero

Lenguaje: C# (.NET — Windows Forms)

Herramienta: Visual Studio

Versión: 1.0

Guatemala, 2026

1. Descripción General del Sistema

El sistema es un juego de estrategia por turnos en tablero desarrollado con el lenguaje de programación C# utilizando la plataforma Windows Forms. La aplicación cuenta con dos versiones integradas en el mismo proyecto: una interfaz gráfica (GUI) construida con formularios de Windows, y una base lógica en consola que sirve como núcleo del juego.

El juego se desarrolla sobre un tablero de 8×8 casillas representado internamente como una matriz bidimensional de objetos Pieza. Dos jugadores se turnan para mover sus piezas (Rey, Torre y Soldado) siguiendo reglas específicas para cada tipo, con el objetivo de capturar el Rey del oponente o eliminar todas sus piezas.

2. Arquitectura del Proyecto

El proyecto se divide en dos espacios de nombres principales que organizan la lógica del sistema:

Namespace	Descripción	Archivos
JuegoProyectoFinal	Contiene toda la lógica del juego y las interfaces gráficas (Windows Forms).	FormMenu, FormLogin, FormJugadores, FormJuego, Juego, Jugador, Pieza, Tablero, Program
JuegoTablero	Contiene las clases de consola (versión base del sistema).	Login, Menu

La aplicación inicia en Program.cs, que lanza el formulario principal FormMenu. A partir de ahí el usuario navega entre los formularios según las acciones que realice.

3. Estructura de Archivos

Archivo	Tipo	Responsabilidad
Program.cs	Punto de entrada	Inicializa la aplicación Windows Forms y lanza FormMenu.
Juego.cs	Lógica principal	Controla el flujo del juego: turnos, movimientos, ataques, puntaje y ganador.
Jugador.cs	Modelo	Representa a un jugador con su identificador y nombre.
Pieza.cs	Modelo	Representa una pieza del tablero con tipo, símbolo y jugador al que pertenece.
Tablero.cs	Modelo	Gestiona la matriz 8×8 e inicializa las posiciones de las piezas.
FormMenu.cs	Vista (GUI)	Menú principal con botones para iniciar, ver reglas, ver puntaje y salir.

Archivo	Tipo	Responsabilidad
FormLogin.cs	Vista (GUI)	Formulario de autenticación con usuario y contraseña oculta.
FormJugadores.cs	Vista (GUI)	Formulario para registrar los nombres de los dos jugadores.
FormJuego.cs	Vista (GUI)	Formulario principal del juego: muestra el tablero, gestiona movimientos y actualiza info.
Login.cs	Consola	Versión de consola del login (clase base de referencia).
Menu.cs	Consola	Versión de consola del menú principal.

4. Descripción de Clases

4.1 Clase Jugador

Namespace: JuegoProyectoFinal | Archivo: Jugador.cs

Representa a uno de los dos jugadores de la partida. Almacena su identificador numérico y su nombre.

Miembro	Tipo	Descripción
id	int (campo público)	Identificador del jugador: 1 para el primero, 2 para el segundo.
nombre	string (campo público)	Nombre ingresado por el jugador al inicio de la partida.
Jugador()	Constructor vacío	Permite instanciar un jugador sin parámetros.
Jugador(int id, string nombre)	Constructor	Crea un jugador con su ID y nombre definidos.

4.2 Clase Pieza

Namespace: JuegoProyectoFinal | Archivo: Pieza.cs

Representa una pieza del tablero. Cada pieza tiene un tipo (Rey, Torre o Soldado), un símbolo visual y el número del jugador al que pertenece.

Miembro	Tipo	Descripción
tipo	string (campo público)	Tipo de pieza: "Rey", "Torre" o "Soldado".
simbolo	char (campo público)	Carácter reservado para representación. El formulario usa símbolos Unicode de ajedrez.
jugador	int (campo público)	Jugador al que pertenece la pieza (1 o 2).

Miembro	Tipo	Descripción
Pieza(string tipo, char simbolo, int jugador)	Constructor	Inicializa todos los atributos de la pieza.

4.3 Clase Tablero

Namespace: JuegoProyectoFinal | Archivo: Tablero.cs

Administra la matriz de 8×8 casillas que representa el tablero. Es la estructura de datos principal del juego.

Miembro	Tipo	Descripción
casillas	Pieza[,] (campo público)	Matriz bidimensional 8×8. Cada celda puede contener una Pieza o null si está vacía.
Inicializar()	void	Limpia el tablero y coloca las piezas en sus posiciones iniciales fijas.

Posiciones iniciales definidas por el sistema:

Pieza	Jugador 1 (Blancas)	Jugador 2 (Negras)
Rey	Fila 7, Columna 4	Fila 0, Columna 4
Torre 1	Fila 7, Columna 0	Fila 0, Columna 0
Torre 2	Fila 7, Columna 7	Fila 0, Columna 7
Soldados (4)	Fila 6, Columnas 2, 3, 4, 5	Fila 1, Columnas 2, 3, 4, 5

4.4 Clase Juego

Namespace: JuegoProyectoFinal | Archivo: Juego.cs

Es el núcleo lógico del sistema. Controla el estado completo de la partida: turno activo, puntajes, selección de pieza, validación y ejecución de movimientos, detección de ataques y verificación de victoria. También mantiene el récord histórico de puntuación.

Miembro	Tipo	Descripción
tablero	Tablero (público)	Instancia del tablero de juego que contiene las piezas.
j1 / j2	Jugador (público)	Objetos que representan a los dos jugadores de la partida.
turno	int (público)	Indica el turno activo: 1 para Jugador 1, 2 para Jugador 2.
puntosJ1 / puntosJ2	int (público)	Acumulan los puntos obtenidos por cada jugador durante la partida.

Miembro	Tipo	Descripción
mejorPuntaje	int (estático)	Récord global de puntuación. Persiste durante toda la sesión.
mejorJugador	string (estático)	Nombre del jugador que tiene el mejor puntaje registrado.
filaSeleccionada / colSeleccionada	int (público)	Coordenadas de la pieza seleccionada en el turno actual.
haySeleccion	bool (público)	Indica si el jugador ya eligió una pieza origen.
Iniciar(string, string)	void	Inicializa una nueva partida: crea jugadores, reinicia tablero y puntajes.
IntentarMover(int, int)	string	Gestiona la selección y el movimiento de piezas. Retorna un mensaje de resultado.
MovimientoValido(Pieza, int, int, int, int)	bool	Valida si el movimiento de una pieza cumple las reglas de su tipo.
CaminoLibre(int, int, int, int)	bool	Verifica que no haya piezas intermedias en la trayectoria de la Torre.
VerificarGanador()	string	Detecta si alguno de los jugadores ganó la partida. Retorna el nombre del ganador o null.

4.5 Clase FormLogin

Namespace: JuegoProyectoFinal | Archivo: FormLogin.cs

Formulario de autenticación que se muestra antes de iniciar una partida. Valida las credenciales del sistema. La contraseña se oculta con el carácter '●' mientras el usuario escribe.

Miembro	Descripción
USUARIO_CORRECTO (constante)	Valor fijo: "admin"
PASSWORD_CORRECTA (constante)	Valor fijo: "Sistemas2026!" (contiene mayúscula, minúscula, número y carácter especial)
txtContrasena.PasswordChar = '●'	Oculta la contraseña en pantalla mientras se escribe
btnEntrar_Click()	Compara las credenciales y cierra el formulario con DialogResult.OK si son correctas; muestra error si no lo son

4.6 Clase FormJugadores

Namespace: JuegoProyectoFinal | Archivo: FormJugadores.cs

Formulario para registrar los nombres de los dos jugadores antes de comenzar la partida. Valida que ambos campos tengan texto antes de continuar.

Propiedad	Descripción
NombreJ1 (string, auto-property)	Almacena el nombre del Jugador 1 (Blancas)
NombreJ2 (string, auto-property)	Almacena el nombre del Jugador 2 (Negras)
BtnIniciar_Click()	Valida que ambos campos no estén vacíos y retorna DialogResult.OK con los nombres

4.7 Clase FormMenu

Namespace: JuegoProyectoFinal | Archivo: FormMenu.cs

Formulario principal que actúa como menú del sistema. Es el primer formulario visible al ejecutar la aplicación. Contiene cuatro botones que corresponden a las opciones del menú requerido.

Botón / Método	Descripción
Iniciar Partida (BtnJugar_Click)	Muestra FormLogin, luego FormJugadores, y si ambos se confirman, inicia FormJuego con una nueva partida.
Ver Reglas (BtnReglas_Click)	Muestra un MessageBox con las reglas del juego.
Mejor Puntaje (BtnPuntaje_Click)	Muestra el récord actual (Juego.mejorJugador y Juego.mejorPuntaje) o el mensaje "Aún no hay puntajes registrados".
Salir	Cierra la aplicación con Application.Exit().
CrearBoton()	Método auxiliar que crea botones con estilo uniforme.

4.8 Clase FormJuego

Namespace: JuegoProyectoFinal | Archivo: FormJuego.cs

Es el formulario principal de la partida. Muestra el tablero gráfico, los campos de entrada de coordenadas, el indicador de turno, los puntajes en tiempo real y los mensajes de juego.

Miembro / Método	Descripción
panelTablero	Panel de dibujo donde se renderiza el tablero 8×8 usando GDI+.
PanelTablero_Paint()	Dibuja las casillas con colores alternos, los símbolos Unicode de las piezas, los números de fila (0-7) en el eje vertical y las letras de columna (A-H) en el eje horizontal.
BtnMover_Click()	Lee las coordenadas en formato 'fila letra' (ejemplo: 6 C), ejecuta todas las validaciones y aplica el movimiento si es válido.

Miembro / Método	Descripción
LetraAColumna(string)	Convierte una letra de columna (A-H) a su índice numérico interno (A=0, B=1, C=2, D=3, E=4, F=5, G=6, H=7). Retorna -1 si la letra es inválida.
ActualizarInfo()	Actualiza la etiqueta de turno con el nombre y símbolo del jugador activo.
ActualizarPuntajes()	Actualiza la etiqueta de puntajes con los puntos actuales de ambos jugadores.
MostrarError(string)	Muestra mensajes de error en rojo sin interrumpir el flujo del juego.
GetSimbolo(Pieza)	Retorna el símbolo Unicode de ajedrez correspondiente al tipo y jugador de la pieza (♔ ♕ ♖ ♗ para J1, ♚ ♛ ♜ ♝ para J2).
BtnReiniciar_Click()	Confirma y reinicia la partida con los mismos jugadores, limpiando el tablero y los puntajes.

4.9 Clases Login y Menu (Consola)

Namespace: JuegoTablero | Archivos: Login.cs, Menu.cs

Son las versiones originales de consola del sistema de login y menú. Forman parte del proyecto como evidencia del desarrollo base en consola según los requisitos técnicos de la propuesta.

Clase	Método	Descripción
Login	IniciarSesion()	Solicita usuario y contraseña por consola. Valida contra las credenciales del sistema.
Menu	Mostrar()	Muestra el menú de 4 opciones en consola y retorna la opción elegida.
Menu	MostrarReglas()	Imprime las reglas del juego en la consola.

5. Lógica de Validación de Movimientos

Toda validación de movimiento ocurre dentro del método `MovimientoValido()` de la clase `Juego`, y en `BtnMover_Click()` de `FormJuego`. Se ejecutan las siguientes comprobaciones en orden:

5.1 Validaciones de entrada (`FormJuego`)

- Las coordenadas se ingresan en formato "fila letra" (ejemplo: 6 C), donde la fila es un número (0-7) y la columna es una letra (A-H).
- La fila debe ser un número entero válido entre 0 y 7.
- La columna debe ser una letra entre A y H (mayúscula o minúscula). El método `LetraAColumna()` la convierte internamente al índice numérico correspondiente (A=0, B=1, ... H=7).
- Debe existir una pieza en la casilla de origen.
- La pieza en origen debe pertenecer al jugador del turno activo.
- La casilla destino no puede estar ocupada por una pieza propia.

5.2 Reglas por tipo de pieza (`Juego.MovimientoValido`)

Pieza	Movimiento permitido	Restricciones
Soldado (J1)	Avanza 1 casilla hacia arriba ($df = -1$, $dc = 0$) si la casilla está vacía.	No puede retroceder. No captura hacia adelante.
Soldado (J2)	Avanza 1 casilla hacia abajo ($df = +1$, $dc = 0$) si la casilla está vacía.	No puede retroceder. No captura hacia adelante.
Soldado (ambos)	Captura en diagonal ($ df =1$, $ dc =1$) si hay pieza rival en el destino.	Solo captura, no se mueve en diagonal sin pieza rival.
Torre	Se mueve en línea recta: horizontal ($df=0$) o vertical ($dc=0$) cualquier número de casillas.	El camino debe estar libre de piezas. Verificado por <code>CaminoLibre()</code> .
Rey	Se mueve 1 casilla en cualquier dirección: $ df \leq 1$ y $ dc \leq 1$.	No puede moverse 2 o más casillas en ninguna dirección.

5.3 Sistema de ataque

Cuando el destino del movimiento contiene una pieza del jugador rival, se considera un ataque. La pieza rival se elimina del tablero y se suman los puntos correspondientes:

- Soldado o Torre capturado: +10 puntos al jugador atacante.
- Rey capturado: +60 puntos (10 base + 50 adicionales).

El sistema muestra un mensaje de captura indicando la pieza eliminada y los puntos obtenidos.

6. Flujo del Juego

El flujo de cada turno sigue este orden exacto según la propuesta del proyecto:

Paso	Acción	Componente responsable
1	Mostrar el tablero con el estado actual.	FormJuego → PanelTablero_Paint()
2	Indicar de quién es el turno.	FormJuego → ActualizarInfo()
3	Solicitar coordenadas de origen y destino.	FormJuego → txtOrigen, txtDestino
4	Validar todos los datos ingresados.	FormJuego → BtnMover_Click() + Juego → MovimientoValido()
5	Ejecutar el movimiento y aplicar captura si corresponde.	FormJuego → BtnMover_Click()
6	Verificar si el juego ha terminado.	Juego → VerificarGanador()
7	Registrar puntaje si hay ganador.	Juego.mejorPuntaje / Juego.mejorJugador (estáticos)

7. Condición de Victoria y Registro de Puntaje

7.1 Condición de victoria

El método VerificarGanador() recorre toda la matriz del tablero y determina si algún jugador ha ganado bajo alguna de estas condiciones:

- El Rey del oponente fue capturado (ya no existe en el tablero).
- El oponente perdió todas sus piezas (no hay ninguna en el tablero).

Si se detecta un ganador, se muestra un MessageBox de victoria y la partida se bloquea.

7.2 Cálculo del puntaje

Pieza capturada	Puntos obtenidos	Detalle
Soldado	10 puntos	Puntos base por captura.
Torre	10 puntos	Puntos base por captura.
Rey	60 puntos	10 puntos base + 50 puntos adicionales por captura del Rey.

7.3 Récord de puntaje

Los campos estáticos Juego.mejorPuntaje y Juego.mejorJugador conservan el récord durante toda la sesión de la aplicación. Al finalizar cada partida:

- Se compara el puntaje del ganador con el récord actual.
- Si es mayor, se actualiza el récord con el nuevo puntaje y el nombre del ganador.
- Si es la primera partida, ese puntaje se guarda automáticamente como el récord inicial.

- Si dos puntajes son iguales, se mantiene el primero registrado.

Desde el menú principal, el botón "Mejor Puntaje" consulta estos campos y muestra el resultado, o el mensaje "Aún no hay puntajes registrados" si no se ha jugado ninguna partida.

8. Sistema de Coordenadas del Tablero

El tablero utiliza un sistema de coordenadas mixto para que el ingreso de movimientos sea más intuitivo, similar al ajedrez tradicional:

Eje	Formato	Valores válidos	Ejemplo
Fila	Número entero	0 al 7 (de arriba hacia abajo)	0 = fila superior, 7 = fila inferior
Columna	Letra	A al H (de izquierda a derecha)	A = columna izquierda, H = columna derecha

Formato de entrada: el jugador escribe la fila y la letra separadas por un espacio. Ejemplo: 6 C (fila 6, columna C).

Conversión interna — método LetraAColumna():

Letra	Índice	Letra	Índice	Letra	Índice	Letra	Índice
A	0	C	2	E	4	G	6
B	1	D	3	F	5	H	7

Si el jugador ingresa una letra fuera del rango A-H, el método retorna -1 y el sistema muestra el error "Columna inválida. Usa A-H." sin ejecutar ningún movimiento. Las letras se aceptan en mayúscula o minúscula.

Representación visual de ejes en el tablero:

Fila \ Col	A	B	C	D	E	F	G	H
0		.	.	.		.	.	
1	.	.					.	.
6	.	.					.	.
7		.	.	.		.	.	

9. Sistema de Login

El sistema cuenta con autenticación en dos capas:

Versión	Clase	Credenciales	Ocultamiento de contraseña
Windows Forms (GUI)	FormLogin	Usuario: admin Contraseña: Sistemas2026!	PasswordChar = '●' (oculta el texto con puntos)
Consola (base)	Login	Usuario: admin Contraseña: Sistemas2026!	Lectura directa por Console.ReadLine()

La contraseña cumple todos los requisitos de seguridad del proyecto:

- Mínimo 8 caracteres: "Sistemas2026!" tiene 13 caracteres.
- Al menos una letra mayúscula: 'S'.
- Al menos una letra minúscula: 'istemas'.
- Al menos un número: '2026'.
- Al menos un carácter especial: '!'.

El sistema no permite acceder al menú del juego hasta que el login sea exitoso. Si las credenciales son incorrectas, se muestra un mensaje de error y se solicitan nuevamente.

10. Requisitos Técnicos Implementados

Requisito	Implementación en el código
Ciclos (for, while)	Tablero.Inicializar() usa ciclos for para limpiar la matriz y colocar soldados. CaminoLibre() usa while para recorrer el camino de la Torre. VerificarGanador() usa for anidado para recorrer toda la matriz.
Estructuras condicionales (if encadenado y anidado)	MovimientoValido() usa if encadenado para evaluar cada tipo de pieza. BtnMover_Click() tiene if anidados para cada validación. VerificarGanador() usa if anidados para detectar piezas y reyes.
Funciones para organizar el código	El sistema está completamente dividido en métodos: Iniciar, IntentarMover, MovimientoValido, CaminoLibre, VerificarGanador, ActualizarInfo, ActualizarPuntajes, PanelTablero_Paint, GetSimbolo, MostrarError, BtnReiniciar_Click.
Matrices para representar el tablero	Tablero.casillas es una matriz Pieza[8,8]. Toda la lógica de posición, movimiento y validación opera sobre esta estructura.
Programación Orientada a Objetos (clases)	El sistema tiene las clases: Jugador, Pieza, Tablero, Juego, FormMenu, FormLogin, FormJugadores, FormJuego, Login, Menu — cada una con responsabilidad específica.
Ejecución continua hasta que el juego termine	El formulario FormJuego permanece abierto y activo hasta que se detecta un ganador o el jugador regresa al menú. El menú principal permanece activo hasta que el usuario elige "Salir".

11. Decisiones de Diseño

Decisión	Justificación
Uso de Windows Forms además de consola	Se implementó la versión gráfica como implementación adicional opcional (sección 23 de la propuesta), manteniendo las clases de consola como base del sistema.
Campos estáticos para el récord (mejorPuntaje, mejorJugador)	Permiten que el récord persista entre partidas durante la sesión sin necesidad de base de datos o archivos externos.
Símbolos Unicode de ajedrez (♔ ♙ ♚ ♜ / ♕ ♖ ♗ ♝)	Hacen el tablero visualmente intuitivo y distinguen claramente entre piezas de cada jugador sin necesidad de colores adicionales.
Renderizado con GDI+ (PanelTablero_Paint)	Permite un tablero fluido, con casillas de colores alternos, sombras en las piezas y borde dorado, sin dependencias externas.
Coordenadas en formato 'fila col'	Formato simple y consistente que cualquier jugador puede entender (ejemplo: "6 3" para fila 6, columna 3).
Validación antes de ejecutar el movimiento	Todas las validaciones se aplican antes de modificar la matriz del tablero, garantizando que el estado del juego siempre sea consistente.

Decisión	Justificación
Selección visual de pieza propia al hacer clic en otra	Si el jugador ingresa coordenadas de otra pieza propia como destino, el sistema redirige automáticamente la selección a esa pieza, mejorando la usabilidad.

12. Herramientas y Tecnologías Utilizadas

Herramienta / Tecnología	Uso en el proyecto
C# (.NET Framework / .NET)	Lenguaje principal de programación del sistema.
Windows Forms (WinForms)	Framework para la interfaz gráfica del juego (formularios, controles, eventos).
Visual Studio	IDE de desarrollo utilizado para escribir, compilar y depurar el código.
GDI+ (System.Drawing)	Biblioteca de gráficos 2D usada para dibujar el tablero y las piezas en FormJuego.
GitHub	Control de versiones y repositorio del proyecto con commits progresivos.